

CONTROLS SYSTEMS

Project 02

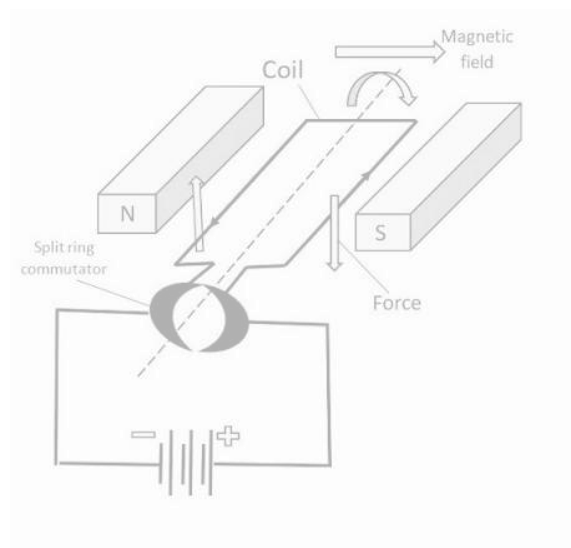
DC MOTOR SPEED CONTROL — MODELING AND DYNAMIC ANALYSIS USING MATLAB

Designed, Developed and Documented by

Zohaib Imtiaz

MATLAB R2024b • Control System Toolbox • Aerospace Engineering

July 2026



Copyright © 2026 Zohaib Imtiaz. All Rights Reserved.

This report is the original work of the author and has been prepared for academic purposes. No part of this document may be reproduced, distributed, or transmitted in any form without proper acknowledgment of the author, except where permitted for educational or research purposes.

Abstract

This project presents the mathematical modeling and dynamic analysis of an armature-controlled DC motor using MATLAB. Starting from first principles, the governing electrical and mechanical equations were derived, and a transfer function relating the applied armature voltage to the output angular velocity was developed. The system was then analyzed through step response, impulse response, and pole analysis. A comprehensive parameter sensitivity study was carried out by varying the armature resistance, rotor inertia, armature inductance, and viscous friction coefficient individually, while keeping all other parameters constant. For each variation, the system poles, transient performance specifications, and steady-state behavior were recorded and physically interpreted. A particularly interesting observation was the discovery of breakaway points in the pole trajectories during the resistance and inertia studies, which connects directly to root locus theory. Finally, the mathematical model was verified using MATLAB's ode45 numerical solver, and the two solutions showed excellent agreement, confirming the validity of the derived transfer function. The project establishes a solid foundation for future controller design and connects directly to aerospace actuator systems used in aircraft, UAVs, and spacecraft.

Table of Contents

Abstract	iii
List of Tables	vii
Table of Figures	viii
1 Objectives	1
2 Introduction.....	2
3 System Description	3
3.1 Electrical Subsystem.....	3
3.2 Mechanical Subsystem.....	3
3.3 Nominal Parameters.....	4
4 Assumptions.....	4
5 Mathematical Modeling.....	5
5.1 Governing Differential Equation	5
5.2 Transfer Function Development	6
6 MATLAB Implementation	8
6.1 MATLAB Implementation	8
6.1.1 MATLAB Functions Used.....	8
6.1.1.1 Transfer Function (tf).....	8
6.1.1.2 Step Response (step).....	8
6.1.1.3 Impulse Response (impulse).....	9
6.1.1.4 Pole Calculation (pole)	9
6.1.1.5 Performance Analysis (stepinfo)	9
6.1.1.6 Numerical Solution (ode45).....	10
6.2 Simulation Procedure.....	10
6.3 Parameter Variation Strategy	10
7 Results and Discussion	11
7.1 Base Case Analysis	11

7.1.1	Base Case Poles:	11
7.1.2	Base Case Performance.....	11
7.1.3	Step Response	12
7.1.4	Impulse Response	12
7.1.5	Observation	13
7.2	Single Parameter Effect	13
7.2.1	Effect of Armature Resistance (R).....	13
7.2.1.1	Pole Movement:	13
7.2.1.2	Step Response	14
7.2.1.3	Performance Specifications:	15
7.2.1.4	Observation	15
7.2.2	Effect of Rotor Inertia (J).....	16
7.2.2.1	Pole Movement:	16
7.2.2.2	Step Response	16
7.2.2.3	Performance Specifications:	17
7.2.2.4	Observation	17
7.2.3	Effect of Armature Inductance (L)	18
7.2.3.1	Pole Movement:	18
7.2.3.3	Performance Specifications:	19
7.2.3.4	Observation	19
7.2.4	Effect of Viscous Friction (b)	20
7.2.4.1	Pole Movement:	20
7.2.4.3	Performance Specifications:	21
7.2.4.4	Observation	21
7.3	Comparative Analysis:.....	22
7.3.1	Resistance vs Friction	22

7.3.2	Inertia vs Inductance	22
7.3.3	ODE45 Verification	23
8	Aerospace Relation	25
9	Conclusions	26
10	Future Work	27
11	Appendix	28
11.1	MATLAB Code	28
11.1.1	Base Case	28
11.1.2	Effect of Armature Resistance (R).....	29
11.1.3	Effect of Rotor Inertia (J).....	29
11.1.4	Effect of Viscous Friction (b)	30
11.1.5	Effect of Armature Inductance (L)	31
11.1.6	Ode45 Solver	31

List of Tables

<i>Table 1 Nominal Plant Parameters</i>	<i>4</i>
<i>Table 2 Base Case System Performance.....</i>	<i>11</i>
<i>Table 3 Effect of Armature Resistance (R) on Poles</i>	<i>13</i>
<i>Table 4 Transient Performance Specifications for Varied Armature Resistance (R)</i>	<i>15</i>
<i>Table 5 Effect of Rotor Inertia (J) on Poles</i>	<i>16</i>
<i>Table 6 Transient Performance Specifications for Varied Rotor Inertia (J)</i>	<i>17</i>
<i>Table 7 Effect of Armature Inductance (L) on Poles</i>	<i>18</i>
<i>Table 8 Transient Performance Specifications for Varied Armature Inductance (L)</i>	<i>19</i>
<i>Table 9 Effect of Viscous Friction (b) on Poles.....</i>	<i>20</i>
<i>Table 10 Transient Performance Specifications for Varied Viscous Friction (b)</i>	<i>21</i>

Table of Figures

Figure 1 Working of a DC Motor	2
Figure 2 Circuit Diagram of DC Motor	3
Figure 3 Motor Step Response - Open Loop	12
Figure 4 DC Motor Impulse Response - Open Loop	12
Figure 5 Effect of Resistance on Speed Response.....	14
Figure 6 Effect of Inertia on Speed Response	16
Figure 7 Effect of Inductance on Speed Response	18
Figure 8 Effect of Friction on Speed Response	20
Figure 9 Ode45 Solver Verification (Overlaid plot).....	23
Figure 10 Ode45 Solver Verification (Side-by-Side Comparison Subplots.....	24

1 Objectives

The primary objective of this project is to model, simulate, and analyze the dynamic behavior of an armature-controlled DC motor using MATLAB and the Control System Toolbox.

The specific objectives are:

- Derive the governing electrical and mechanical equations of the DC motor from physical principles.
- Apply Kirchhoff's Voltage Law and Newton's Rotational Law to obtain two coupled differential equations.
- Take the Laplace transform and derive the transfer function relating input voltage $V(s)$ to output angular velocity $\omega(s)$.
- Implement the transfer function in MATLAB and analyze the open-loop step response, impulse response, and pole locations.
- Investigate how each motor parameter (**R, J, L, b**) individually affects the transient performance and steady-state speed.
- Verify the transfer function model using the ode45 numerical solver.
- Establish the connection between DC motor actuator dynamics and aerospace engineering applications.

2 Introduction

DC motors are among the most widely used electromechanical devices in modern engineering. They are found in robotics, industrial automation, electric vehicles, and aerospace systems, converting electrical energy into controlled rotational motion. In aerospace engineering specifically, DC motors serve as actuators — devices that translate controller commands into physical movement. Servo motors actuate aircraft control surfaces, reaction wheel motors rotate spacecraft for attitude control, and BLDC motors drive UAV propellers.

Understanding the dynamic behavior of a DC motor is therefore not just an academic exercise, it is a fundamental requirement for anyone designing control systems for real aerospace hardware. A DC motor has two coupled subsystems: an electrical side (governed by resistance, inductance, and back-EMF) and a mechanical side (governed by rotor inertia and viscous friction). The interaction between these two sides determines how fast the motor responds, how much it overshoots, and what final speed it reaches for a given input voltage.

In this project, the DC motor is treated as a control system plant. The mathematical model is built from scratch using Newton's and Kirchhoff's laws, the transfer function is derived by hand and verified in MATLAB, and the effects of each physical parameter are investigated systematically. This approach mirrors exactly how professional control engineers approach a new actuator before designing a controller for it.

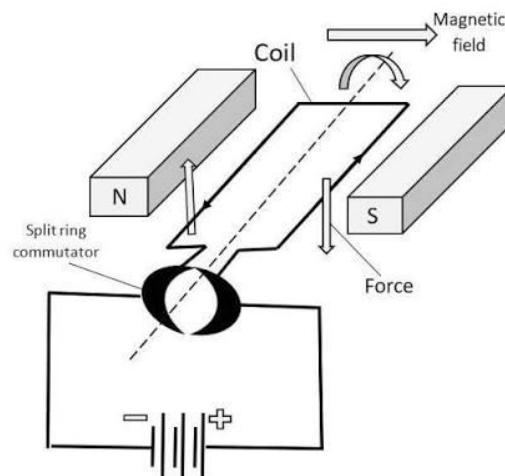


Figure 1 Working of a DC Motor

3 System Description

The DC motor studied in this project is an armature-controlled permanent magnet DC motor.

It consists of two coupled subsystems:

3.1 Electrical Subsystem

The electrical subsystem includes the armature resistance R (Ω), armature inductance L (H), the applied supply voltage $V(t)$, and the back electromotive force (back-EMF) generated by the rotating armature. When a voltage is applied, current flows through the armature circuit and produces electromagnetic torque. As the motor speeds up, the back-EMF increases and opposes the supply voltage, which naturally limits the armature current and prevents the motor from accelerating indefinitely.

3.2 Mechanical Subsystem

The mechanical subsystem includes the rotor moment of inertia J ($\text{kg}\cdot\text{m}^2$), the viscous friction coefficient b ($\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$), and the angular velocity ω of the rotor. The electromagnetic torque produced by the armature current accelerates the rotor, while friction continuously opposes the motion. At steady state, the motor reaches an equilibrium where the driving torque exactly balances the frictional resistive torque, and the angular velocity becomes constant.

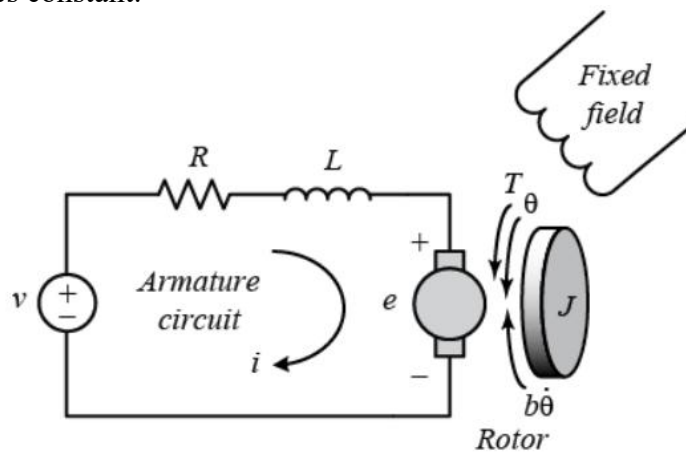


Figure 2 Circuit Diagram of DC Motor

3.3 Nominal Parameters

The following nominal parameter values were used throughout the base-case analysis:

Table 1 Nominal Plant Parameters

Parameter	Symbol	Value	Unit
Moment of Inertia	J	0.01	$\text{kg}\cdot\text{m}^2$
Viscous Friction	b	0.1	$\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$
Armature Resistance	R	1	Ω
Armature Inductance	L	0.5	H
Torque Constant	K_t	0.01	$\text{N}\cdot\text{m}/\text{A}$
Back-EMF Constant	K_e	0.01	$\text{V}\cdot\text{s}/\text{rad}$

4 Assumptions

The following assumptions were made to simplify the analysis while preserving the dominant dynamic behavior of the motor:

1. All motor parameters remain constant and do not change with temperature or operating conditions.
2. Magnetic saturation effects in the armature are neglected.
3. Armature reaction is ignored.
4. Friction is purely viscous and proportional to angular velocity.
5. No external mechanical load torque is applied. The motor runs under no-load conditions.
6. The motor operates within its linear region throughout all simulations.

5 Mathematical Modeling

5.1 Governing Differential Equation

1. The Electrical System (KVL)

Applying Kirchhoff's Voltage Law (KVL) to the armature circuit loop:

$$V(t) = V_R(t) + V_L(t) + e(t)$$

Where:

- $V_R = i(t)R$ (Voltage drop across armature resistance)
- $V_L = L \frac{di(t)}{dt}$ (Voltage drop across armature inductance)
- $e(t) = K_e \omega(t) = K_e \frac{d\theta(t)}{dt}$ (Back EMF produced by the rotating coil)

Substituting these values into the KVL equation yields **Equation (1)**:

$$L \frac{di(t)}{dt} + Ri(t) = V(t) - K_e \frac{d\theta(t)}{dt}$$

2. The Mechanical System (Newton's 2nd Law)

The sum of torques acting on the rotor inertia must equal its angular acceleration:

$$\sum T = J\alpha(t) = J \frac{d\omega(t)}{dt} = J \frac{d^2\theta(t)}{dt^2}$$

The acting torques are:

- **Driving Motor Torque (T_m):** Proportional to armature current, $T_m = K_t i(t)$
- **Opposing Friction Torque (T_f):** Due to bearing viscous damping, $T_f = b\omega(t) = b \frac{d\theta(t)}{dt}$

Setting up the balance equation:

$$T_{\text{driving}} - T_{\text{opposing}} = J \frac{d\omega(t)}{dt}$$

$$K_t i(t) - b\omega(t) = J \frac{d\omega(t)}{dt}$$

Rearranging gives us **Equation (2)** in terms of angular velocity (ω):

$$J \frac{d\omega(t)}{dt} + b\omega(t) = K_t i(t)$$

Or substituting $\omega(t) = \frac{d\theta(t)}{dt}$ to express it in terms of angular position (θ):

$$J \frac{d^2\theta(t)}{dt^2} + b \frac{d\theta(t)}{dt} = K_t i(t)$$

5.2 Transfer Function Development

Transforming to the Laplace Domain (s -Domain)

Assuming **zero initial conditions** ($i(0) = 0, \theta(0) = 0, \dot{\theta}(0) = 0$), we apply the Laplace Transform ($\frac{d}{dt} \rightarrow s$):

From the **Electrical Equation (1)**:

$$LsI(s) + RI(s) = V(s) - K_e s\Theta(s)$$

$$(Ls + R)I(s) = V(s) - K_e s\Theta(s) \quad \text{--- (3)}$$

From the **Mechanical Equation (2)** (using the position form):

$$Js^2\Theta(s) + bs\Theta(s) = K_t I(s)$$

$$s(Js + b)\Theta(s) = K_t I(s)$$

Isolate current $I(s)$ so we can substitute it out:

$$I(s) = \frac{s(Js + b)\Theta(s)}{K_t} \quad \text{--- (4)}$$

Algebraic Substitution & Simplification

Substitute Equation (4) into Equation (3):

$$(Ls + R) \left[\frac{s(Js + b)\Theta(s)}{K_t} \right] = V(s) - K_e s\Theta(s)$$

To eliminate the fraction, multiply the entire equation by K_t :

$$s(Js + b)(Ls + R)\Theta(s) = K_t V(s) - K_t K_e s\Theta(s)$$

Move all terms involving $\Theta(s)$ to the left-hand side:

$$s(Js + b)(Ls + R)\Theta(s) + K_t K_e s\Theta(s) = K_t V(s)$$

Factor out $s\Theta(s)$ on the left side:

$$[(Js + b)(Ls + R) + K_t K_e]s\Theta(s) = K_t V(s)$$

Final Transfer Functions:

A. Angular Position Transfer Function: $\frac{\Theta(s)}{V(s)}$

Isolating $\Theta(s)$ relative to input voltage $V(s)$:

$$\frac{\Theta(s)}{V(s)} = \frac{K_t}{s[(Js + b)(Ls + R) + K_t K_e]}$$

B. Angular Velocity Transfer Function: $\frac{\Omega(s)}{V(s)}$

Since angular velocity is the derivative of position, its Laplace relationship is $\Omega(s) = s\Theta(s)$. Substituting this directly into our last factored step gives:

$$[(Js + b)(Ls + R) + K_t K_e]\Omega(s) = K_t V(s)$$

Dividing both sides yields the velocity transfer function:

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{(Js + b)(Ls + R) + K_t K_e}$$
$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_t K_e)}$$

6 MATLAB Implementation

6.1 MATLAB Implementation

6.1.1 MATLAB Functions Used

Several built-in MATLAB functions from the Control System Toolbox were used throughout this project. Each function served a specific purpose in the overall analysis.

6.1.1.1 Transfer Function (tf)

The tf function was used to construct mathematical models directly from the derived transfer functions. The numerator and denominator coefficients corresponding to each system configuration were entered into MATLAB to create Linear Time-Invariant (LTI) models suitable for analysis.

```
num = Kt;  
den = [(J*L) (J*R+b*L) (b*R+Kt*Ke)];  
G = tf(num,den);
```

This command generates the transfer function

$$G(s) = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_tK_e)}$$

which forms the basis for all subsequent simulations.

6.1.1.2 Step Response (step)

The step function was used to study the response of each system when subjected to a unit step input. This represents the application of a constant force that remains active throughout the simulation.

The response provides valuable information regarding

- Rise Time
- Peak Time
- Overshoot
- Settling Time

- Steady State Value
- Overall Stability

By comparing the step responses of different configurations, the influence of mass, damping, and spring stiffness on system performance was investigated.

6.1.1.3 Impulse Response (impulse)

The impulse function was used to analyze the response of each system to a unit impulse input. Unlike the step response, the impulse input represents a force applied over an extremely short duration before being removed.

Impulse responses reveal the natural dynamics of the system and are particularly useful for observing oscillatory behavior, damping characteristics, and energy dissipation.

6.1.1.4 Pole Calculation (pole)

The pole command was used to determine the locations of the poles of each transfer function.

```
p = pole(G);
```

The pole locations provide direct insight into system stability and transient behavior. Systems with complex conjugate poles exhibit oscillatory responses, whereas real poles typically produce non-oscillatory behavior. Throughout the project, the pole locations were compared with the corresponding response plots to establish the relationship between mathematical models and physical system behavior.

6.1.1.5 Performance Analysis (stepinfo)

The `stepinfo` function was employed to calculate important transient response specifications automatically.

The calculated parameters include

- Rise Time
- Peak Time

- Maximum Overshoot
- Settling Time
- Peak Value
- Steady-State Value

These numerical values were used to support the graphical observations obtained from the step response plots and to compare the performance of different system configurations quantitatively.

6.1.1.6 Numerical Solution (ode45)

To verify the analytical transfer-function approach, MATLAB's ode45 solver was applied to the complete mass-spring-damper system.

The governing second-order differential equation was rewritten as a system of first-order differential equations before being solved numerically.

Unlike transfer-function analysis, which assumes a linear system represented in the Laplace domain, ode45 directly integrates the governing differential equations in the time domain. Comparing the numerical and analytical solutions provided additional confidence in the correctness of the developed mathematical model.

6.2 Simulation Procedure

6.3 Parameter Variation Strategy

To understand the influence of each physical parameter independently, only one parameter was varied at a time while the remaining parameters were held constant whenever possible.

The investigation included variations in

- | | |
|--------------|-----------|
| • Resistance | Inertia |
| • Inductance | Friction. |

For each parameter study, a for-loop was used to study the step response at different varying conditions of a single parameter. The pole locations and step info results were printed to the command window for each case and recorded for analysis.

7 Results and Discussion

7.1 Base Case Analysis

With all parameters at nominal values, the transfer function produces a second-order system. The pole analysis immediately tells us everything about how this motor will behave before even looking at a plot.

7.1.1 Base Case Poles:

$$p = \begin{matrix} -9.9975 \\ -2.0025 \end{matrix}$$

Both poles are real and negative, sitting on the negative real axis of the complex plane. This means the system is overdamped — there will be no oscillations, no overshoot, and the motor will smoothly accelerate to its final speed. The pole at -9.9975 is far from the origin, so it decays very quickly and has almost no visible effect on the response. The pole at -2.0025 is the dominant pole — it is closest to the origin and governs how slowly or quickly the motor reaches steady state.

7.1.2 Base Case Performance

Table 2 Base Case System Performance

Parameter	Value
Rise Time	1.135 s
Peak Time	3.676 s
Settling Time	2.065 s
Overshoot	0%
Steady-State Speed	0.0998 rad/s

7.1.3 Step Response

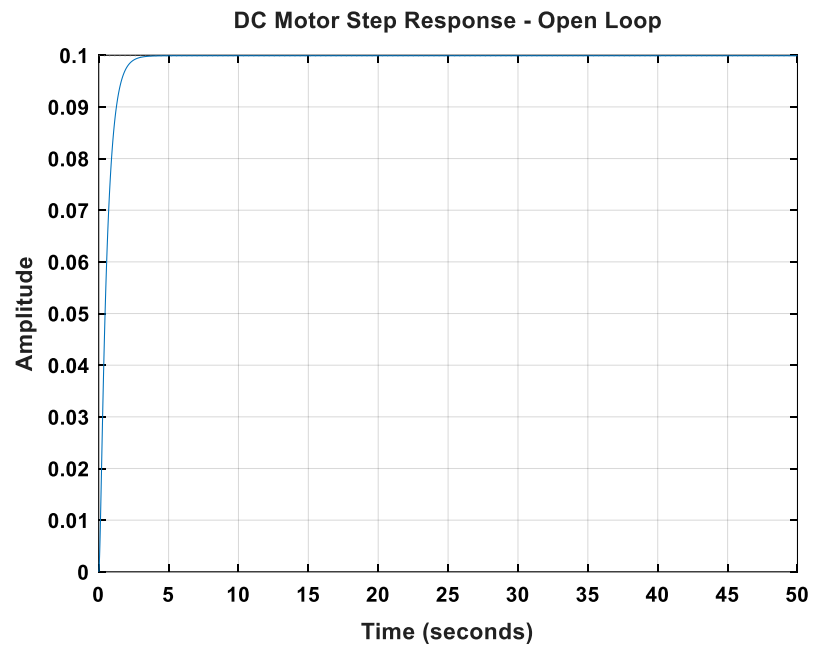


Figure 3 Motor Step Response - Open Loop

7.1.4 Impulse Response

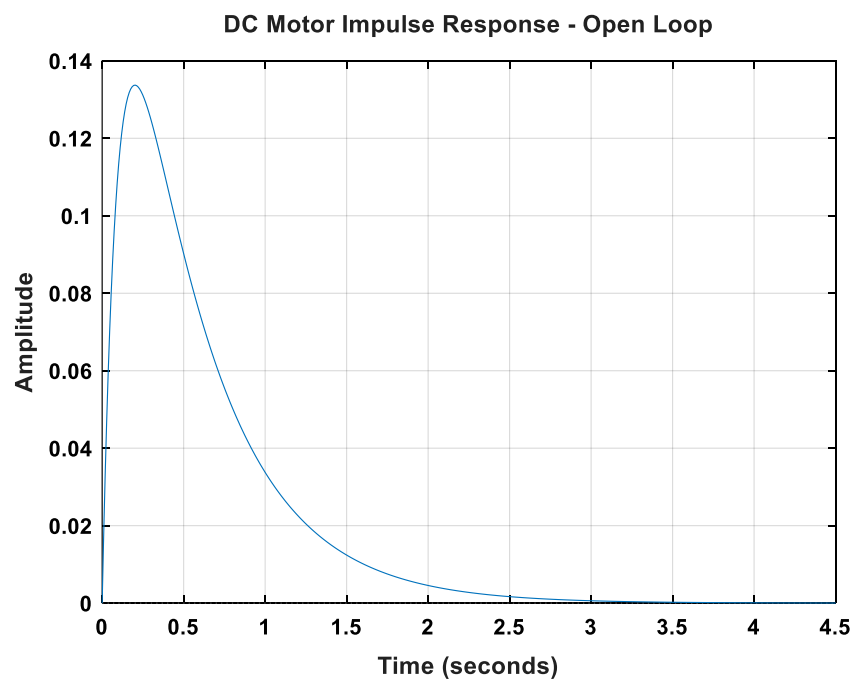


Figure 4 DC Motor Impulse Response - Open Loop

7.1.5 Observation

When a 1 V step input is applied, the motor achieves a steady-state angular velocity of ≈ 0.10 rad/s. This matches the system's theoretical steady-state gain:

$$\frac{K_t}{bR + K_t K_e} = \frac{0.01}{(0.1 \times 1) + (0.01 \times 0.01)} \approx \mathbf{0.0998}$$

This outcome underscores a key open-loop characteristic: the output velocity depends purely on fixed hardware parameters rather than user intent. To overcome this limitation and ensure precise speed tracking, a closed-loop PID architecture is introduced in Project 03.

7.2 Single Parameter Effect

7.2.1 Effect of Armature Resistance (R)

The armature resistance varied from $0.5 \, \Omega$ to $5 \, \Omega$ while all other parameters were held constant. Resistance appears in both the electrical time constant (L/R) and the steady-state gain expression, so it affects both the speed of response and the final speed reached.

7.2.1.1 Pole Movement:

Table 3 Effect of Armature Resistance (R) on Poles

R (Ω)	Pole 1	Pole 2	Nature
0.5	-9.9978	-1.0022	Real — overdamped
1.0	-9.9975	-2.0025	Real — overdamped
2.0	-9.9967	-4.0033	Real — overdamped
5.0	$-10.000 \pm 0.1414j$	—	Complex — underdamped (breakaway)

As resistance increases from $0.5\ \Omega$ to $2\ \Omega$, both poles remain real and negative but gradually move closer to each other on the real axis. At $R = 5\ \Omega$, something interesting happens — the two real poles meet and then split into a complex conjugate pair. This is called a breakaway point in root locus theory, and it was discovered here through direct observation rather than formal root locus analysis. This shows that the system transitions from overdamped to underdamped behavior as resistance crosses a critical value.

7.2.1.2 Step Response

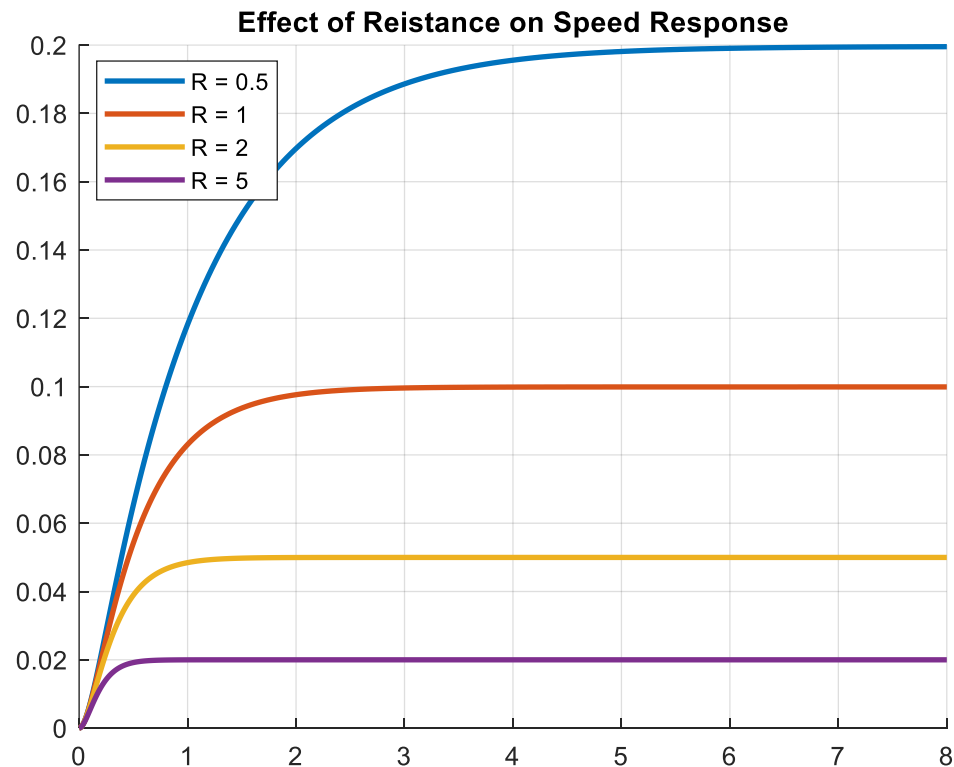


Figure 5 Effect of Resistance on Speed Response

7.2.1.3 Performance Specifications:

Table 4 Transient Performance Specifications for Varied Armature Resistance (R)

R (Ω)	Rise Time (s)	Peak Time (s)	Settling Time (s)	Steady-State (rad/s)
0.5	2.210	7.361	4.009	higher
1.0	1.135	3.676	2.065	0.0998
2.0	0.616	2.110	1.105	lower
5.0	0.336	1.193	0.583	much lower

7.2.1.4 Observation

As resistance increases, the steady-state speed decreases because a larger fraction of the supply voltage is lost as heat across the resistor (I^2R loss), leaving less effective voltage to drive the motor. Because the final speed is lower, the motor takes less time to reach it — so counterintuitively, higher resistance produces faster settling times despite reducing performance. This is not a desirable trade-off; it simply means the motor is running slower and getting there faster because the target is lower.

7.2.2 Effect of Rotor Inertia (J)

Rotor inertia represents the rotational mass of the motor shaft. A heavier rotor resists changes in angular velocity more strongly, exactly like a heavier object resists linear acceleration. Inertia was varied from 0.005 to 0.1 kg·m².

7.2.2.1 Pole Movement:

Table 5 Effect of Rotor Inertia (J) on Poles

J (kg·m ²)	Pole 1	Pole 2	Nature
0.005	-19.998	-2.002	Real — overdamped
0.010	-9.998	-2.003	Real — overdamped
0.050	$-2.000 \pm 0.063j$	—	Complex — breakaway point
0.100	-1.998	-1.002	Real — overdamped again

7.2.2.2 Step Response

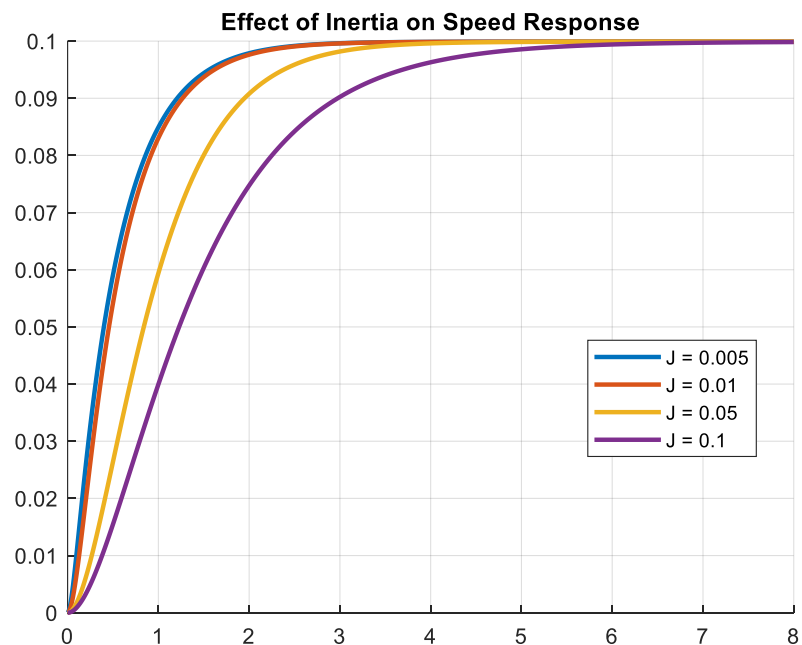


Figure 6 Effect of Inertia on Speed Response

7.2.2.3 Performance Specifications:

Table 6 Transient Performance Specifications for Varied Rotor Inertia (J)

J (kg·m ²)	Rise Time (s)	Peak Time (s)	Settling Time (s)	Steady-State (rad/s)
0.005	1.106	3.680	2.007	0.0998
0.010	1.135	3.676	2.065	0.0998
0.050	1.677	5.964	2.913	0.0998
0.100	2.587	7.767	4.594	0.0998

7.2.2.4 Observation

Unlike resistance, inertia does not affect the steady-state speed at all — the final speed remains 0.0998 rad/s across all four cases. This is because inertia only appears in the numerator of the mechanical time constant, not in the steady-state gain formula. What inertia does affect is how long the motor takes to reach that speed. A heavier rotor (higher J) takes longer to spin up, which is exactly what Newton's second law predicts. The breakaway point occurs at $J \approx 0.05$ kg·m², where the poles briefly become complex before returning to the real axis.

7.2.3 Effect of Armature Inductance (L)

Inductance is the electrical equivalent of inertia — just as inertia resists changes in velocity, inductance resists changes in current. Higher inductance means the armature current builds up more slowly after voltage is applied, which delays torque production and slows the initial motor response.

7.2.3.1 Pole Movement:

Table 7 Effect of Armature Inductance (L) on Poles

L (H)	Pole 1	Pole 2	Nature
0.01	-99.989	-10.011	Real — very fast decay
0.10	$-10.000 \pm 0.316j$	—	Complex — breakaway point
0.50	-9.9975	-2.0025	Real — overdamped (nominal)
1.00	-9.9989	-1.0011	Real — overdamped

7.2.3.2 Step Response

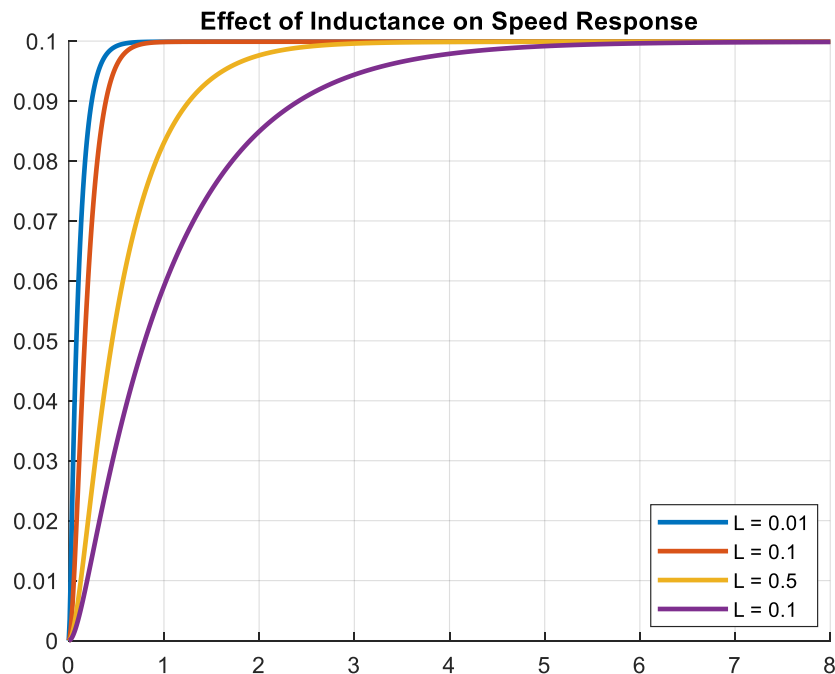


Figure 7 Effect of Inductance on Speed Response

7.2.3.3 Performance Specifications:

Table 8 Transient Performance Specifications for Varied Armature Inductance (L)

L (H)	Rise Time (s)	Peak Time (s)	Settling Time (s)	Steady-State (rad/s)
0.01	0.221	0.736	0.401	0.0998
0.10	0.335	1.193	0.583	0.0998
0.50	1.135	3.676	2.065	0.0998
1.00	2.213	7.360	4.013	0.0998

7.2.3.4 Observation

Just like inertia, inductance does not affect the steady-state speed — 0.0998 rad/s is reached in all four cases. Inductance purely affects the transient response. The smaller the inductance, the faster the current builds, the faster torque is produced, and the faster the motor accelerates. The breakaway point occurs at $L \approx 0.10$ H, showing the same pole collision behavior observed in the resistance and inertia studies.

7.2.4 Effect of Viscous Friction (b)

Viscous friction is the mechanical resistance that opposes rotation. It acts like a velocity-proportional brake — the faster the motor spins, the stronger the braking force. Friction was varied from 0.01 to 1.0 N·m·s/rad.

7.2.4.1 Pole Movement:

Table 9 Effect of Viscous Friction (b) on Poles

b (N·m·s/rad)	Pole 1	Pole 2	Nature
0.01	-1.980	-1.020	Real — close together
0.10	-9.998	-2.003	Real — overdamped (nominal)
0.50	-49.999	-2.000	Real — one pole far left
1.00	-100.000	-2.000	Real — very separated

7.2.4.2 Step Response

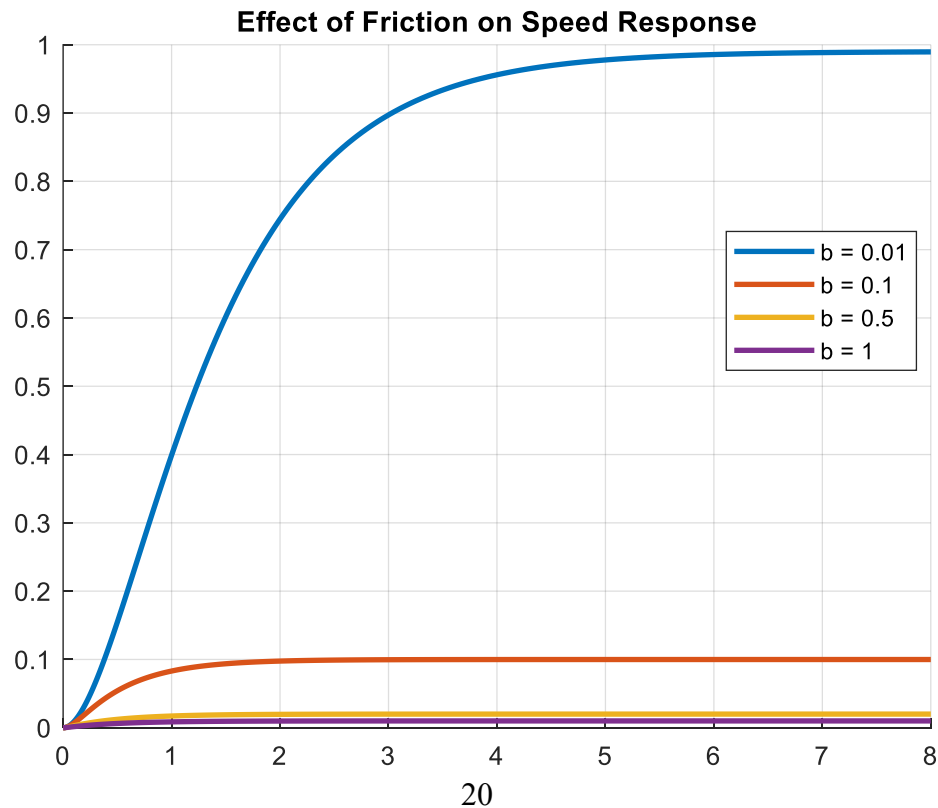


Figure 8 Effect of Friction on Speed Response

7.2.4.3 Performance Specifications:

Table 10 Transient Performance Specifications for Varied Viscous Friction (b)

b (N·m·s/rad)	Rise Time (s)	Peak Time (s)	Settling Time (s)	Steady-State (rad/s)
0.01	2.558	7.627	4.538	high
0.10	1.135	3.676	2.065	0.0998
0.50	1.099	3.660	1.976	lower
1.00	1.099	3.661	1.966	much lower

7.2.4.4 Observation

Friction behaves similarly to resistance in that it reduces the steady-state speed as it increases — more friction means more torque is consumed just to keep the motor spinning, leaving less net torque to accelerate the rotor. However, the settling time behavior with friction is more nuanced than with resistance. At very low friction ($b = 0.01$), the system is lightly damped and takes a long time to settle. As friction increases, damping improves and settling time decreases. But unlike the resistance case, the steady-state speed drop is the primary concern with excessive friction.

7.3 Comparative Analysis:

7.3.1 Resistance vs Friction

An interesting cross-comparison emerged during the parameter study. When $R = 0.5 \Omega$ and $b = 0.5 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$ were compared, the rise time, peak time, and settling time for the resistance case were approximately double those for the friction case at the same numerical value. This difference arises because R affects the electrical time constant (L/R) and appears in both the numerator and denominator of the gain expression, giving it a stronger combined influence on the overall response.

7.3.2 Inertia vs Inductance

Similarly, when $J = 0.5 \text{ kg}\cdot\text{m}^2$ and $L = 0.5 \text{ H}$ were compared, the settling time for the inertia case was noticeably longer than for the inductance case. This is because inertia directly determines the mechanical time constant (J/b), which has a more dominant influence on the overall speed response than the electrical time constant alone. In practical motor selection for aerospace actuators, this means reducing rotor inertia gives faster response than reducing inductance for the same numerical reduction in parameter value.

7.3.3 ODE45 Verification

To verify that the derived transfer function correctly represents the physical differential equations, the motor state equations were solved numerically using MATLAB's ode45 solver. The two state variables were the armature current $i(t)$ and the angular velocity $\omega(t)$. The governing equations implemented were:

$$\begin{aligned} di/dt &= (V - K_e \omega - R i) / L \\ d\omega/dt &= (K_t i - b \omega) / J \end{aligned}$$

The ode45 solution for angular velocity was plotted alongside the transfer function step response on the same graph. The two curves overlapped almost perfectly throughout the entire simulation period — during the initial transient, through the rise, and at steady state. The small numerical differences that appeared were due to the adaptive step-size integration tolerance of ode45 and are entirely negligible for engineering purposes.

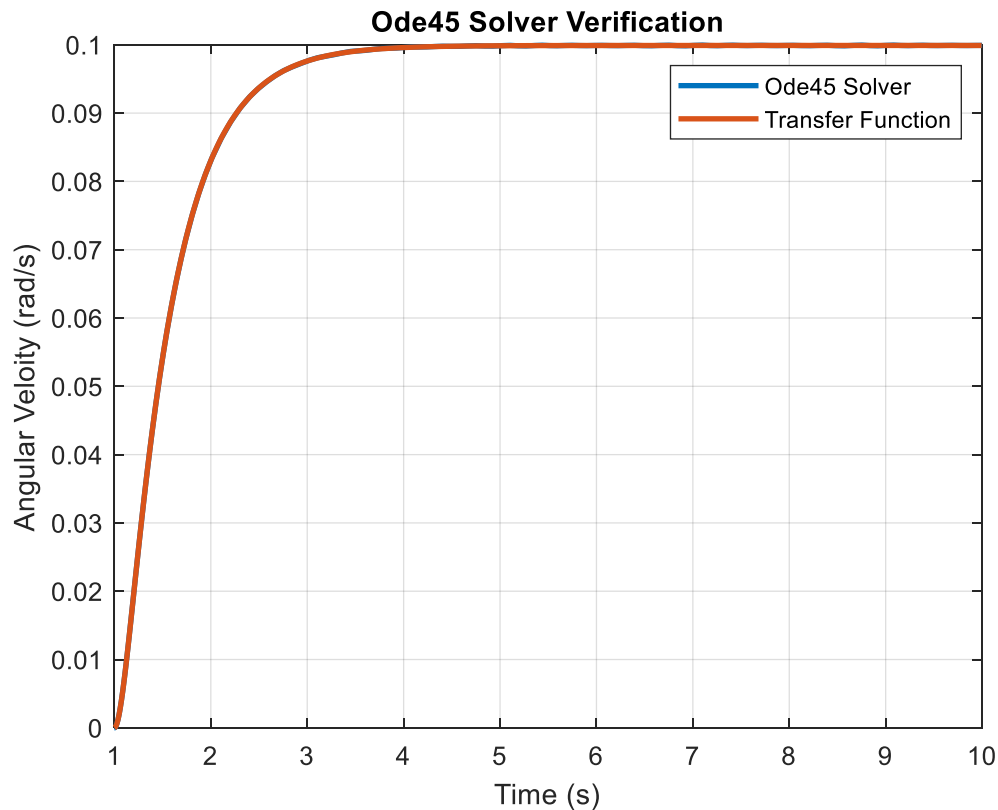


Figure 9 Ode45 Solver Verification (Overlaid plot)

Ode45 Solver Verification

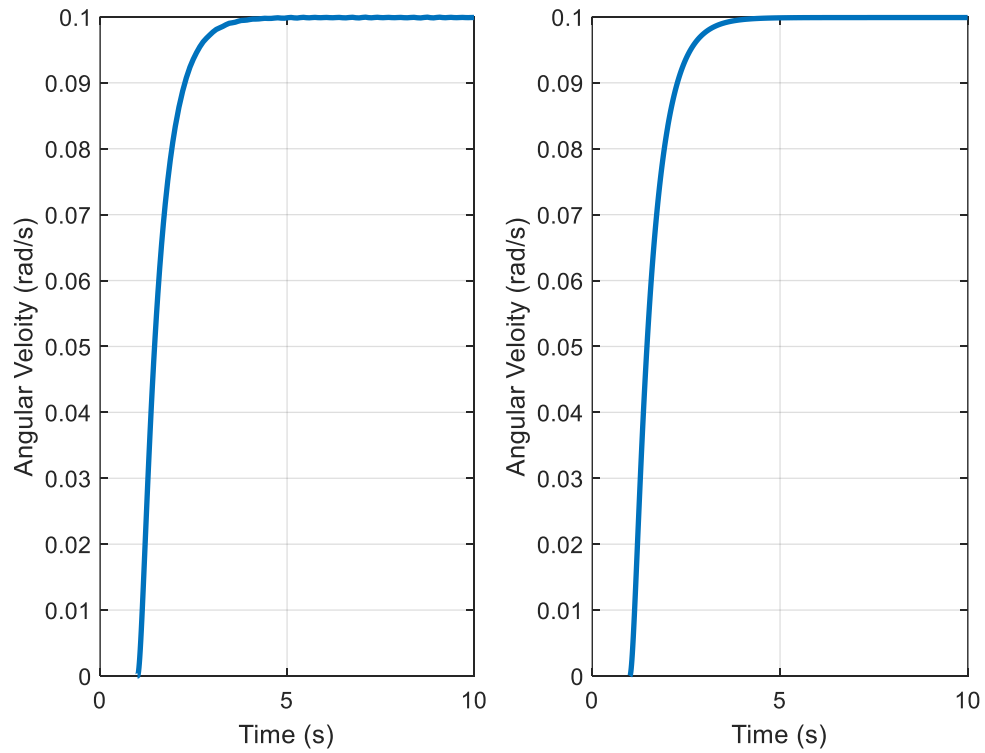


Figure 10 Ode45 Solver Verification (Side-by-Side Comparison Subplots)

This verification is significant because it confirms that the Laplace-domain transfer function, derived through algebraic manipulation of the original differential equations, accurately captures the same physics that ode45 integrates directly. The model is validated and can be used with confidence for controller design in Project 03.

8 Aerospace Relation

The DC motor model developed in this project connects directly to several real aerospace engineering applications. In aerospace, controllers do not directly move control surfaces or rotate spacecraft — they send commands to actuators, and actuators do the physical work. Understanding actuator dynamics is therefore essential for designing any closed-loop aerospace control system.

In **fixed-wing aircraft**, servo motors driven by DC actuators move elevator, aileron, and rudder control surfaces. The transient response of the actuator directly affects the speed at which the aircraft can respond to pilot commands or autopilot corrections. A slow actuator (high inertia or high resistance) creates lag in the control loop, which can lead to instability in aggressive maneuvers.

In **UAVs and multirotor drones**, BLDC motors — which are DC motors with electronic commutation — provide all thrust and attitude control. Understanding how inertia and friction affect speed response is critical for designing stable flight controllers. The parameter sensitivity analysis from this project directly informs motor selection decisions for UAV propulsion.

In **satellites**, reaction wheels use DC motors to produce torques for attitude control without consuming propellant. A satellite might need to rotate 30 degrees in a controlled, smooth maneuver. The motor dynamics studied here determine how quickly and how accurately that rotation can be achieved. High rotor inertia in a reaction wheel is actually desirable in some cases — it stores more angular momentum for attitude control purposes.

The key engineering conclusion from this project is that the open-loop DC motor cannot reliably track a desired speed command — its steady-state speed depends on parameter values, not on what the control system wants. This is the fundamental motivation for Project 03, where a PID controller will be added to create a closed-loop system capable of reaching and maintaining any commanded speed regardless of parameter variations.

9 Conclusions

This project successfully completed the mathematical modeling and open-loop dynamic analysis of an armature-controlled DC motor using MATLAB. The following conclusions were drawn from the analysis:

The derived transfer function

$$G(s) = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_tK_e)}$$

correctly describes the relationship between applied voltage and output angular velocity. This was verified by confirming that the ode45 numerical solution and the transfer function step response are identical.

The base case system is overdamped with two real negative poles at -9.9975 and -2.0025 . The dominant pole at -2.0025 governs the speed of the transient response. The system settles to 0.0998 rad/s in approximately 2.07 seconds with zero overshoot.

Armature resistance and viscous friction both reduce the steady-state speed as they increase, because they consume energy that would otherwise drive rotation. Rotor inertia and armature inductance do not affect the steady-state speed but slow down the transient response.

A breakaway point — where the two real poles meet and briefly become complex — was observed at $R \approx 5 \Omega$, $J \approx 0.05 \text{ kg}\cdot\text{m}^2$, and $L \approx 0.1 \text{ H}$. This is a root locus phenomenon discovered experimentally before any formal root locus study.

The open-loop motor reaches only 0.0998 rad/s for a 1 V input. If this motor were driving an aircraft control surface actuator or a satellite reaction wheel, this behavior would be unacceptable — the system cannot track a commanded speed without a controller. This motivates the design of a PID closed-loop controller in Project 03.

10 Future Work

Project 02 focused entirely on open-loop plant analysis. The natural next step is Project 03, which will take the exact same DC motor transfer function and add a PID controller to create a closed-loop speed control system. The following extensions are planned:

1. Design a PID controller for the DC motor and compare P, PI, PD, and PID responses on the same plant.
2. Tune PID gains manually and compare with MATLAB's PID tune automatic result.
3. Build the closed-loop system in Simulink using Transfer Function and PID Controller blocks.
4. Perform root locus analysis to understand how controller gain affects closed-loop pole locations.
5. Conduct frequency response analysis using Bode plots to determine gain and phase margins.
6. Develop a state-space model of the DC motor for modern control techniques.
7. Apply the LQR (Linear Quadratic Regulator) and compare with PID performance.
8. Extend to an aircraft pitch controller using the same modeling approach.

11 Appendix

11.1 MATLAB Code

11.1.1 Base Case

```
clc , clear all , close all

%% -----DC MOTOR SPEED CONTROL-----

%% ----- System Parameters-----
J = 0.01 ; % Moment of Inertia (kg.m^2)
b = 0.1 ; % Viscous Friction Coefficient (N.m.s)
R = 1 ; % Resistane (Ohm)
L = 0.5 ; % Inuctance (H)
Kt = 0.01; % Motor Torque Constant (N.m/A)
Ke = 0.01; % Back-EMF Constant (V/rad/s)

% Transfer Function
num = Kt ;
den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
G = tf(num,den)

% Poles of System
disp("Poles:")
p = pole(G)

%% ----- Response of System -----
figure(1)
% Step Response
t = [0 50];
step(G,t)
disp("System Performance:")
S = stepinfo(G)
title('DC Motor Step Response - Open Loop')

figure(2)
% Impulse Response
impulse(G)
title('DC Motor Impulse Response - Open Loop')
```

```
%% ----- Single Parameter Effect -----
```

```
t = 0:0.01:8;
R = 1 ; J = 0.01 ; b = 0.1 ; L = 0.5 ; Kt = 0.01; Ke = 0.01;
```

11.1.2 Effect of Armature Resistance (R)

```
%% Effect of Resistance
figure(3)
R_val = [0.5, 1, 2, 5];
for i = 1:length(R_val)
    R = R_val(i);
    num = Kt;
    den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
    G_R = tf(num,den);
    [y,x] = step(G_R,t) ; hold on
    plot(x,y,'LineWidth',2)
    fprintf('Poles at R = %.2f \n',R)
    p_R = pole(G_R)
    info_R = stepinfo(G_R);
    fprintf('System Performance at R = %.2f : \n',R)
    fprintf('    Rise Time : %.3f s \n',info_R.RiseTime)
    fprintf('    Peak Time : %.3f s \n',info_R.PeakTime)
    fprintf('    Settling Time : %.3f s \n',info_R.SettlingTime)
    fprintf('    Overshoot Time : %.3f \n',info_R.Overshoot)
    grid on
end
legend('R = 0.5' , 'R = 1' , 'R = 2' , 'R = 5',Location='best')
title ('Effect of Resistance on Speed Response')
```

11.1.3 Effect of Rotor Inertia (J)

```
%% Effect of Moment of Inertia
figure(4)
J_val = [0.005, 0.01, 0.05, 0.1];
% J_val = [0.005, 0.01,0.02,0.03,0.04, 0.05, 0.1,0.2,0.3, 0.4 ,
0.5 , 0.6 , 0.7 , 0.8 ,0.9 ,1];
for i = 1:length(J_val)
    J = J_val(i);
    num = Kt;
    den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
    G_J = tf(num,den);
    [y,x] = step(G_J,t) ; hold on
    plot(x,y,'LineWidth',2)
```

```

fprintf('Poles at J = %.3f \n',J)
p_J = pole(G_J)
info_J = stepinfo(G_J);
fprintf('System Performance at J = %.2f : \n',J)
fprintf('    Rise Time : %.3f s \n',info_J.RiseTime)
fprintf('    Peak Time : %.3f s \n',info_J.PeakTime)
fprintf('    Settling Time : %.3f s \n',info_J.SettlingTime)
fprintf('    Overshoot Time : %.3f \n',info_J.Overshoot)
grid on
end
legend('J = 0.005' , 'J = 0.01' , 'J = 0.05' , 'J =
0.1',Location='best')
title ('Effect of Inertia on Speed Response')

```

11.1.4 Effect of Viscous Friction (b)

```

%% Effect of Friction
figure(5)
b_val = [0.01, 0.1, 0.5, 1];
% b_val = [0.01,0.02,0.03,0.04,0.05,0.06,0.07,0.08,0.09, 0.1,
0.5, 1];
for i = 1:length(b_val)
    b = b_val(i);
    num = Kt;
    den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
    G_b = tf(num,den);
    [y,x] = step(G_b,t) ; hold on
    plot(x,y,'LineWidth',2)
    fprintf('Poles at b = %.2f \n',b)
    p_b = pole(G_b)
    info_b = stepinfo(G_b);
    fprintf('System Performance at b = %.2f : \n',b)
    fprintf('    Rise Time : %.3f s \n',info_b.RiseTime)
    fprintf('    Peak Time : %.3f s \n',info_b.PeakTime)
    fprintf('    Settling Time : %.3f s \n',info_b.SettlingTime)
    fprintf('    Overshoot Time : %.3f \n',info_b.Overshoot)
    grid on
end
legend('b = 0.01' , 'b = 0.1' , 'b = 0.5' , 'b =
1',Location='best')
title ('Effect of Friction on Speed Response')

```

11.1.5 Effect of Armature Inductance (L)

```
% Effect of Inductance
figure(6)
L_val = [0.01, 0.1, 0.5, 1];
% L_val = [0.01,
0.02,0.03,0.04,0.05,0.06,0.07,0.08,0.09,0.1,0.2,0.3,0.4, 0.5, 1];
for i = 1:length(L_val)
    L = L_val(i);
    num = Kt;
    den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
    G_L = tf(num,den);
    [y,x] = step(G_L,t) ; hold on
    plot(x,y,'LineWidth',2)
    fprintf('Poles at L = %.2f \n',L)
    p_L = pole(G_L)
    info_L = stepinfo(G_L);
    fprintf('System Performance at L = %.2f : \n',L)
    fprintf('    Rise Time : %.3f s \n',info_L.RiseTime)
    fprintf('    Peak Time : %.3f s \n',info_L.PeakTime)
    fprintf('    Settling Time : %.3f s \n',info_L.SettlingTime)
    fprintf('    Overshoot Time : %.3f \n',info_L.Overshoot)
    grid on
end
legend('L = 0.01' , 'L = 0.1' , 'L = 0.5' , 'L =
0.1',Location='best')
title ('Effect of Inductance on Speed Response')
```

11.1.6 Ode45 Solver

```
clc ; clear all ; close all ;

% Ode45 Solver Code
R = 1 ; J = 0.01 ; b = 0.1 ; L = 0.5 ; Kt = 0.01; Ke = 0.01;
V = 1 ;
f = @(t,X) [(V-Ke*X(2)-R*X(1))/L;
            (Kt*X(1)-b*X(2))/J];
t_Span = [1 10];
X0 = [0 0];

[t1,X1] = ode45(f,t_Span,X0);

figure(1)
plot(t1,X1(:,2),'LineWidth',2) ; hold on;
grid on
```

```

% Transfer Function
num = Kt;
den = [(J*L) (J*R + b*L) b*R + Kt*Ke];
G = tf(num,den);

[X2,t2] = step(G,t_Span);
plot(t2,X2,'LineWidth',2)

xlabel("Time (s)");
ylabel("Angular Velocity (rad/s)")
legend("Ode45 Solver", "Transfer Function")
title("Ode45 Solver Verification")

figure(2)
subplot(1,2,1)
plot(t1,X1(:,2),'LineWidth',2)
xlabel("Time (s)");
ylabel("Angular Velocity (rad/s)")
grid on
subplot(1,2,2)
plot(t2,X2,'LineWidth',2)
grid on
xlabel("Time (s)");
ylabel("Angular Velocity (rad/s)")
sgtitle("Ode45 Solver Verification")

```